Agentic Execution Graph (AEG) Framework

Version: 1.0 (Draft)

Core Principle: Deterministic Orchestration of Probabilistic Tasks.

1. Executive Summary

The AEG Framework is a distributed architecture designed to move beyond "autonomous" agents that rely on raw LLM reasoning for flow control. Instead, it treats LLMs as functional utilities within a **Strictly Gated Execution Graph**. By separating Planning, Evaluation, and Execution into isolated, horizontally scalable units, the framework ensures that complex multi-step tasks are predictable, auditable, and resilient.

---

2. System Architecture

A. The Cognitive Layer (The Planner)

- **Role:** Transforms high-level user intent into a **Domain Specific Language (DSL)** plan.
- **Output:** An `Execution_Graph` consisting of **Nodes** (Tasks) and **Edges** (Transitions).
- **Identity Generation:** Generates a unique `PlanID`.
- **State Mapping:** Defines the initial state and expected terminal state.

B. The Regulatory Layer (The Plan Evaluator)

- **Role:** Acts as a "Pre-flight Check." It performs static analysis on the DSL.
- **Validation Rules:**
    - **Graph Theory:** Checks for cycles (unless explicitly allowed), orphaned nodes, and reachability.
    - **Constraint Checking:** Ensures the plan fits within token/cost/time budgets.
    - **Tool Feasibility:** Verifies that all `TaskID`s in the graph correspond to registered tools in the `Executor Registry`.
- **Approval:** Emits an `ApprovalToken` required by the Orchestrator to begin.

C. The Operational Layer (Detached Orchestrators)

- **Instance Mapping:** 1:1 ratio. Each `PlanID` spawns exactly one ephemeral `OrchestratorInstance`.

- **Orchestrator ID:** Mirrors the PlanID.
- **Workflow:** 1. Loads the approved DSL.

    2. Invokes **Workers** for each active node.

    3. Evaluates **Edge Logic** post-execution to decide the next transition.

    4. Self-destructs upon reaching a Terminal Node or TTL (Time-to-Live).

---

3. The Identity & State Ledger

To ensure absolute traceability, the framework enforces a **Composite ID Schema**:

Identity Level	Format	Example
Root Plan	{PlanID}	research-001
Node Instance	{PlanID}.{NodeID}	research-001.web_search
Execution Attempt	{PlanID}.{NodeID}.{vN}	research-001.web_search.v2

The Ledger Protocol

All state changes are written to a persistent, append-only **State Ledger** indexed by PlanID. This allows any "Resume" operation to look up the last successful NodeInstance and restart from that exact edge.

---

4. The DSL Concept (Logic on Edges)

The "secret sauce" of the framework is that **the Edge is the Guard**. Instead of a node deciding where to go next, the edge evaluates the node's output against a boolean condition.

Conceptual DSL Snippet:

YAML

```
nodes:
  - id: "query_db"
    tool: "sql_executor"
    params: { "query": "SELECT..." }

edges:
  - from: "query_db"
    to: "process_results"
    condition: "count(node.output) > 0" # Deterministic logic
  - from: "query_db"
    to: "handle_empty"
    condition: "count(node.output) == 0"
```

5. Blind Spot Mitigation (Built-in)

1. **Orphaned Orchestrators:** A global **Reaper Service** monitors the Orchestrator Registry and terminates instances exceeding their TTL or resource quota.
2. **Logic Deadlocks:** The **Plan Evaluator** rejects graphs that do not have an "Else" or "Error" branch for every conditional node.
3. **Context Poisoning:** The DSL enforces **Input/Output Filtering**. Nodes only receive the specific keys from the Ledger they are subscribed to, preventing context window overflow.

---

6. Proposed Development Roadmap

1. **Phase 1:** Define the **DSL Specification** (YAML/JSON schema).
2. **Phase 2:** Build the **Orchestrator Factory** (The 1:1 spawning mechanism).
3. **Phase 3:** Develop the **Condition Engine** (How edges evaluate node outputs).
4. **Phase 4:** Implement the **Planner & Evaluator** LLM-wrappers.